

REPORT CS 343 LAB 4

Group 38

- Raj Nandani Kumari: 230123048
- Neha Sellappan Devadas: 230123040
- Subhashree Kedia: 230123063
- Aditya Samal: 230123002

Our code files:  CS343_Lab_4

Task 1:

Task Overview

We have to implement a more efficient version of the fork() system call Copy-on-Write (COW):

- Implement Reference Counting for Physical Pages
- Modify fork() for COW
- Implement a Page Fault Handler for COW

Methodology and Code Overview

Reference Counting for Physical Pages

Track how many page table entries point to each physical page, so we know when it is safe to free a page.

- ❖ Global Reference Counter Array:
 - Added a global array (`int refcnt[PHYSTOP/PGSIZE]`) to hold the reference count for each physical page.
 - Protected this array with a spinlock (`struct spinlock refcnt_lock`).
- ❖ `kalloc()` Changes:
 - When a new page is allocated, set its reference count to 1.
- ❖ `kfree()` Changes:
 - When freeing a page, decrement its reference count.
 - Only free the page (add to the free list) if the reference count drops to 0.
- ❖ Helper Functions:
 - Implemented `incred(p)` and `decred(p)` to safely increment/decrement the reference count for a physical address, using the spinlock for thread safety.

Used these helpers in all places where a page is shared or released (fork, unmapping, page fault, etc.).

Modify fork() to Use COW

Make fork efficient by sharing all pages between parent and child, only copying on write.

❖ **uvmcopy()** Changes:

- Instead of allocating new pages and copying data, map the parent's physical pages into the child's page table.

```
mem = kalloc();           // allocate new physical page
memmove(mem, (char*)pa, PGSIZE); // copy parent page data
mappages(new, i, PGSIZE, (uint64)mem, flags); // map into child
```

< Earlier implementation

```
if(flags & PTE_W){
    flags &= ~PTE_W; // Child: remove write bit
    *pte &= ~PTE_W; // Parent: remove write bit too
}
mappages(new, i, PGSIZE, pa, flags); // map same physical address
krefinc(pa); // increment reference count
```

< COW implementation

❖ For each mapped page:

- Removed the **PTE_W** flag from parent and child PTEs, making them read-only.
- Incremented the reference count for the shared physical page.

❖ Page Table Synchronization:

- Ensured both parent and child see the page as read-only, so any write triggers a page fault.

❖ Error Handling:

- If mapping fails, clean up by unmapping any pages already mapped and decrementing reference counts.

Implement COW Page Fault Handler

When a process tries to write to a shared (read-only) page, allocate a new page and copy the data (copy-on-write).

❖ Trap Handler (**usertrap()** in **trap.c**):

- Detects store (write) page faults (**scause == 15**).
- Gets the faulting virtual address from the **stval** register.
- Locate the PTE for the faulting address.
- Check if the page is present, user-accessible, and not writable.

❖ Reference Count Check:

- If the reference count for the physical page is greater than 1:
 - Allocated a new physical page.
 - Copy the contents from the old page to the new page.
 - Update the PTE to point to the new page and set the **PTE_W** flag.
 - Decrement the reference count of the old page.
- If the reference count is 1:
 - Just set the **PTE_W** flag in the PTE (no need to copy).
- Resume Execution:
 - After handling, return from the trap so the user instruction is retried and now succeeds.

```

else if((r_scause() == 15 || r_scause() == 13)) {
// Page fault - try demand paging first
if(handle_page_fault(p, r_stval()) == 0) {
// Successfully handled by demand paging
} else if(r_scause() == 15 && cowfault(r_stval()) == 0) {
// Try COW fault handling for store page faults
} else if(vmfault(p->pagetable, r_stval(), (r_scause() == 13)? 1 : 0) == 0) {
// Try lazy allocation
} else {
printf("Page fault failed pid=%d va=0x%lx scause=0x%lx\n", p->pid, r_stval(), r_scause());
setkilled(p);
}
}

```

When a page fault occurs (**scause == 13 or 15**):

Detect Page Fault

- ❖ **13**: Load (read) fault — accessing unmapped memory.
- ❖ **15**: Store (write) fault — writing to unmapped or read-only memory.

Demand Paging **handle_page_fault()**

- ❖ Loads the missing page from disk/file only when accessed.
- ❖ Saves memory and speeds up process startup.

Copy-on-Write **cowfault()**

- ❖ Triggered only on store faults (**scause == 15**).
- ❖ Handles writes to shared (read-only) pages after **fork()**.
- ❖ Allocates a private copy and makes it writable.

Lazy Allocation **vmfault()**

- ❖ Allocates a new physical page when a process first accesses unallocated memory (e.g., heap/stack).
- ❖ Supports efficient memory growth via **sbrk()**.

If all fail → Kill Process

- ❖ If none of the handlers succeed, the fault is invalid → process is terminated.

Update Memory Unmapping (uvmunmap) and Exit

Ensure reference counts are decremented when a process unmaps or exits.

- ❖ **uvmunmap()** Changes:
 - When unmapping a page, decremented its reference count.
 - Only free the physical page if the reference count drops to 0.
- ❖ Process Exit:
 - When a process exits, unmap all its pages, properly updating reference counts.

Test Code:

```

int main() {
    char *p = sbrk(4096);
    if(p == (char*)-1) {
        printf("sbrk failed\n");
        exit(1);
    }
    p[0] = 'A';
    int pid = fork();
    if(pid < 0){
        printf("fork failed\n");
        exit(1);
    }
}

```

```

}
if(pid == 0){
    // child
    p[0] = 'C'; // should cause COW
    if(p[0] != 'C'){
        printf("child write failed (%c)\n", p[0]);
        exit(1);
    }
    exit(0);
} else {
    wait(0);
    if(p[0] != 'A'){
        printf("parent sees wrong value %c (COW broken)\n", p[0]);
        exit(1);
    }
    printf("Copy-on-Write Fork OK\n");
}
exit(0);
}

```

- This code has first allocated a page using **sbrk command**, the **parent** process has initially written **A** on the page.
- The process then creates a child and waits for child's execution to complete to continue its own execution. (using **Copy on Write Fork**)
- The **child** gets a **shared copy of the page allocated** by parent and overwrites **C** on the page which causes creation of **copy of the page** for the child.
- The child's execution gets over and the parent continues, it sees the same **A** it had written initially on the original page making sure that Copy on Write Fork works.

Output:

```

hart 1 starting
hart 2 starting
init: starting sh
$ cowtest
Copy-on-Write Fork OK

```

As it is seen, directly "Copy-on-Write Fork OK" is printed which means COW fork worked exactly as we expected.

Task 2:

Task Overview

We had to extend the xv6 operating system to support demand paging using a Most Recently Used (MRU) page replacement policy.:

- Modify the kernel to support swapping of user pages
- Add new system calls to monitor paging activity

Methodology and Code Overview

All of the below mentioned logic is present in file > [mru.c](#)

Designing the MRU List

To track all resident user pages in Most Recently Used (MRU) order, so the most recently accessed page can be evicted when memory is full.

Implementation:

- ❖ Created a global doubly-linked list in the kernel to represent the MRU list.
- ❖ Each node should stores:
 - Process ID (pid)
 - Virtual address (va)
 - Pointer to the page table entry (PTE) or physical address
 - Pointers to previous and next nodes
 - Protect the MRU list with a spinlock for concurrency.

Updating the MRU List on Access

On every user page access (detected via page faults, and optionally in copyin/copyout).

- ❖ When a page is accessed, moved its node to the head of the MRU list.
- ❖ When a new page is allocated or swapped in, inserted it at the head of the MRU list.
- ❖ When a page is evicted, removed its node from the list.

Page Replacement Policy (Eviction)

- ❖ Trigger:
 - When kalloc() failed (no free physical pages), initiated eviction.
- ❖ Victim Selection:
 - Selected the head of the MRU list as the victim.
- ❖ Eviction Steps:
 - Removed the victim node from the MRU list.
 - Swapped the page out to the process's swap file.
 - Updated the process's metadata to record the swap location.
 - Cleared the PTE and marked the page as swapped out (with a custom flag in the PTE).

(A swap file is a file on disk used by the operating system to temporarily store pages of a process's memory when there is not enough physical RAM available.)

Swapping Mechanism

- ❖ Swap-Out:
 - Wrote the contents of the evicted page to the process's swap file at a free offset.
 - Updated the process's swap table to record which virtual address is swapped out and its swap file offset.
- ❖ Swap-In:
 - On a page fault, if the faulting address is marked as swapped out, read the page from the swap file into a free physical page.
 - Update the PTE to point to the new physical page and mark it as present.
 - Inserted the page at the head of the MRU list.
- ❖ Swap File Management:

- Each process should have its own swap file.

Implement helper functions for reading/writing pages to/from the swap file.

Page Fault Handling

On Page Fault:

In `usertrap()` (in `trap.c`), checked if the faulting address is:

- ❖ Not present and not swapped out: terminate the process (invalid access).
- ❖ Swapped out: perform swap-in as described above.
- ❖ Present but not writable (COW): handle as in COW logic.

After handling, update the MRU list to move the accessed page to the head.

System Calls

`getpagestat(int pid, struct pagestat st):`

- ❖ Defined a new struct `pagestat`.
- ❖ Added counters for page faults, swap-ins, and swap-outs to struct `proc`.
- ❖ Implemented the syscall to fill the `pagestat` struct for the given PID.
- ❖ Updated counters in the page fault handler and swap routines.

`dumpmru(void):`

- ❖ Implemented a syscall that prints the MRU list to the console, showing process IDs and virtual page numbers in MRU order.
- ❖ Traversed the MRU list and print each entry.

Register the new syscalls in `syscall.c`, `syscall.h`, `sysproc.c`, `user.h`, and any other relevant files.

Demo/Test Code:

Initialization: Defines `PAGES_TO_ALLOC = 15` and `TEST_ITERATIONS = 50`. Declares arrays for page addresses, last write tracking, and page statistics.

Page Allocation:

```
printf("Allocating %d pages...\n", PAGES_TO_ALLOC);
for(i = 0; i < PAGES_TO_ALLOC; i++) {
    memory[i] = sbrk(4096); // Allocate one page
```

Allocates 15 pages using `sbrk(4096)` (4 KB each). Ensures enough memory is used to exceed physical RAM and cause swapping.

Access Pattern:

```
for(i = 0; i < TEST_ITERATIONS; i++) {
    // Access pages in a pattern that will trigger MRU behavior
    // First half: sequential access, second half: random access
    int page_idx;
    if(i < TEST_ITERATIONS/2) {
        page_idx = i % PAGES_TO_ALLOC; // Sequential access
    } else {
        // Random-ish access pattern to trigger more swapping
        page_idx = (i * 7 + 3) % PAGES_TO_ALLOC;
    }
}
```

First 25 iterations > sequential page access. Next 25 iterations > random-like access using `(i*7+3)%PAGES_TO_ALLOC`. Each access involves writing and reading data from the page to simulate usage and cause page faults.

Periodic Statistics: Every 5 iterations, `getpagestat()` fetches current stats (faults, swap-ins, swap-outs). `dumpmru()` prints the MRU list showing which pages are most recently used.

```

// Track this write
last_write[page_idx] = i;

// Write some data to the page
for(j = 0; j < 4096; j += 64) {
    memory[page_idx][j] = (char)(i + j);
}

// Read some data from the page
char sum = 0;
for(j = 0; j < 4096; j += 64) {
    sum += memory[page_idx][j];
}

printf("Iteration %d: Accessed page %d, checksum = %d\n", i, page_idx, sum);

if((i + 1) % 5 == 0) {
    if(getpagestat(getpid(), &stats) == 0) {
        printf(" Page Statistics: Faults=%d, Swap-ins=%d, Swap-outs=%d\n",
            stats.page_faults, stats.swap_ins, stats.swap_outs);
    }

    printf(" Current MRU List:\n");
    dumpmru();
}

```

Data Verification: After all accesses, each page's contents are checked against expected values. Detects any data corruption caused during swapping.

```

int errors = 0;
for(i = 0; i < PAGES_TO_ALLOC; i++) {
    if(last_write[i] >= 0) {
        // Check a few offsets
        int check_count = 0;
        for(j = 0; j < 256; j += 64) {
            char expected = (char)(last_write[i] + j);
            if(memory[i][j] != expected) {
                printf("ERROR: Data corruption in page %d offset %d! Expected %d, got %d (last write: iter %d)\n",
                    i, j, expected, memory[i][j], last_write[i]);
                errors++;
                if(check_count++ > 3) break; // Don't spam too many errors
            }
        }
        if(check_count == 0) {
            printf("Page %d: Data integrity verified (last write: iter %d)\n", i, last_write[i]);
        }
    }
}
}

```

Output:

```
Iteration 12: Accessed page 12, checksum = 0
Iteration 13: Accessed page 13, checksum = 64
Iteration 14: Accessed page 14, checksum = 128
Page Statistics: Faults=17, Swap-ins=0, Swap-outs=7
Current MRU List:
MRU List (Most Recently Used first):
0: PID=5 VA=0x12000
1: PID=5 VA=0x11000
2: PID=5 VA=0x10000
3: PID=5 VA=0xf000
4: PID=5 VA=0xe000
5: PID=5 VA=0xd000
6: PID=5 VA=0xc000
7: PID=5 VA=0xb000
Total pages in MRU list: 8 (max per process: 8)

Iteration 15: Accessed page 0, checksum = 192
Iteration 16: Accessed page 1, checksum = 0
Iteration 17: Accessed page 2, checksum = 64
Iteration 18: Accessed page 3, checksum = 128
Iteration 19: Accessed page 4, checksum = 192
Page Statistics: Faults=22, Swap-ins=5, Swap-outs=12
Current MRU List:
MRU List (Most Recently Used first):
0: PID=5 VA=0x8000
1: PID=5 VA=0x7000
2: PID=5 VA=0x6000
3: PID=5 VA=0x5000
4: PID=5 VA=0x4000
5: PID=5 VA=0x12000
6: PID=5 VA=0x11000
7: PID=5 VA=0x10000
Total pages in MRU list: 8 (max per process: 8)

Iteration 20: Accessed page 5, checksum = 0
Iteration 21: Accessed page 6, checksum = 64
Iteration 22: Accessed page 7, checksum = 128
Iteration 23: Accessed page 0, checksum = 192
Iteration 24: Accessed page 9, checksum = 0
Page Statistics: Faults=27, Swap-ins=10, Swap-outs=17
Current MRU List:
MRU List (Most Recently Used first):
0: PID=5 VA=0xd000
1: PID=5 VA=0xc000
2: PID=5 VA=0xb000
3: PID=5 VA=0xa000
4: PID=5 VA=0x9000
5: PID=5 VA=0x8000
6: PID=5 VA=0x7000
7: PID=5 VA=0x6000
Total pages in MRU list: 8 (max per process: 8)

Iteration 25: Accessed page 13, checksum = 64
Iteration 26: Accessed page 5, checksum = 128
Iteration 27: Accessed page 12, checksum = 192
Iteration 28: Accessed page 4, checksum = 0
Iteration 29: Accessed page 11, checksum = 64

$ mru mem
MRU Memory Test Program
Allocating 15 pages...
Allocated page 0 at address 0x4000
Allocated page 1 at address 0x5000
Allocated page 2 at address 0x6000
Allocated page 3 at address 0x7000
Allocated page 4 at address 0x8000
Allocated page 5 at address 0x9000
Allocated page 6 at address 0xa000
Allocated page 7 at address 0xb000
Allocated page 8 at address 0xc000
Allocated page 9 at address 0xd000
Allocated page 10 at address 0xe000
Allocated page 11 at address 0xf000
Allocated page 12 at address 0x10000
Allocated page 13 at address 0x11000
Allocated page 14 at address 0x12000

Starting random memory access test...
Iteration 0: Accessed page 0, checksum = 0
Iteration 1: Accessed page 1, checksum = 64
Iteration 2: Accessed page 2, checksum = 128
Iteration 3: Accessed page 3, checksum = 192
Iteration 4: Accessed page 4, checksum = 0
Page Statistics: Faults=7, Swap-ins=0, Swap-outs=0
Current MRU List:
MRU List (Most Recently Used first):
0: PID=5 VA=0x0000
1: PID=5 VA=0x7000
2: PID=5 VA=0x6000
3: PID=5 VA=0x5000
4: PID=5 VA=0x4000
5: PID=5 VA=0x14000
Total pages in MRU list: 6 (max per process: 8)

Iteration 5: Accessed page 5, checksum = 64
Iteration 6: Accessed page 6, checksum = 128
Iteration 7: Accessed page 7, checksum = 192
Iteration 8: Accessed page 8, checksum = 0
Iteration 9: Accessed page 9, checksum = 64
Page Statistics: Faults=12, Swap-ins=0, Swap-outs=2
Current MRU List:
MRU List (Most Recently Used first):
0: PID=5 VA=0x0000
1: PID=5 VA=0xc000
2: PID=5 VA=0xb000
3: PID=5 VA=0xa000
4: PID=5 VA=0x9000
5: PID=5 VA=0x8000
6: PID=5 VA=0x7000
7: PID=5 VA=0x6000
Total pages in MRU list: 8 (max per process: 8)

Iteration 10: Accessed page 10, checksum = 128
Iteration 11: Accessed page 11, checksum = 192

Page Statistics: Faults=30, Swap-ins=13, Swap-outs=20
Current MRU List:
MRU List (Most Recently Used first):
0: PID=5 VA=0xf000
1: PID=5 VA=0x10000
2: PID=5 VA=0x11000
3: PID=5 VA=0xd000
4: PID=5 VA=0xc000
5: PID=5 VA=0xb000
6: PID=5 VA=0xa000
7: PID=5 VA=0x9000
Total pages in MRU list: 8 (max per process: 8)

Iteration 30: Accessed page 3, checksum = 128
Iteration 31: Accessed page 10, checksum = 192
Iteration 32: Accessed page 2, checksum = 0
Iteration 33: Accessed page 9, checksum = 64
Iteration 34: Accessed page 1, checksum = 128
Page Statistics: Faults=34, Swap-ins=17, Swap-outs=24
Current MRU List:
MRU List (Most Recently Used first):
0: PID=5 VA=0x5000
1: PID=5 VA=0x6000
2: PID=5 VA=0xe000
3: PID=5 VA=0x7000
4: PID=5 VA=0xf000
5: PID=5 VA=0x10000
6: PID=5 VA=0x11000
7: PID=5 VA=0xd000
Total pages in MRU list: 8 (max per process: 8)

Iteration 35: Accessed page 8, checksum = 192
Iteration 36: Accessed page 0, checksum = 0
Iteration 37: Accessed page 7, checksum = 64
Iteration 38: Accessed page 14, checksum = 128
Iteration 39: Accessed page 6, checksum = 192
Page Statistics: Faults=39, Swap-ins=22, Swap-outs=29
Current MRU List:
MRU List (Most Recently Used first):
0: PID=5 VA=0xa000
1: PID=5 VA=0x12000
2: PID=5 VA=0xb000
3: PID=5 VA=0x4000
4: PID=5 VA=0xc000
5: PID=5 VA=0x2000
6: PID=5 VA=0x0000
7: PID=5 VA=0xe000
Total pages in MRU list: 8 (max per process: 8)

Iteration 40: Accessed page 13, checksum = 0
Iteration 41: Accessed page 5, checksum = 64
Iteration 42: Accessed page 12, checksum = 128
Iteration 43: Accessed page 4, checksum = 192
Iteration 44: Accessed page 11, checksum = 0
Page Statistics: Faults=44, Swap-ins=27, Swap-outs=34
Current MRU List:
MRU List (Most Recently Used first):
0: PID=5 VA=0x3000
1: PID=5 VA=0x0000
2: PID=5 VA=0x6000
3: PID=5 VA=0xe000
4: PID=5 VA=0x7000
5: PID=5 VA=0xf000
6: PID=5 VA=0x8000
7: PID=5 VA=0x10000
Total pages in MRU list: 8 (max per process: 8)

Final verification - checking data integrity...
Page 0: Data integrity verified (last write: iter 36)
Page 1: Data integrity verified (last write: iter 49)
Page 2: Data integrity verified (last write: iter 47)
Page 3: Data integrity verified (last write: iter 45)
Page 4: Data integrity verified (last write: iter 43)
Page 5: Data integrity verified (last write: iter 41)
Page 6: Data integrity verified (last write: iter 39)
Page 7: Data integrity verified (last write: iter 37)
Page 8: Data integrity verified (last write: iter 35)
Page 9: Data integrity verified (last write: iter 48)
Page 10: Data integrity verified (last write: iter 46)
Page 11: Data integrity verified (last write: iter 44)
Page 12: Data integrity verified (last write: iter 42)
Page 13: Data integrity verified (last write: iter 40)
Page 14: Data integrity verified (last write: iter 38)

Final Page Statistics:
Page Faults: 59
Swap-ins: 42
Swap-outs: 49

Final MRU List:
MRU List (Most Recently Used first):
0: PID=5 VA=0x12000
1: PID=5 VA=0x11000
2: PID=5 VA=0x10000
3: PID=5 VA=0xf000
4: PID=5 VA=0xe000
5: PID=5 VA=0xc000
6: PID=5 VA=0xb000
7: PID=5 VA=0xa000
Total pages in MRU list: 8 (max per process: 8)

✓ MRU Memory Test PASSED!
MRU Memory Test Completed!
```

Observations:

- The program successfully allocates **15 pages** (each 4 KB) using **sbrk(4096)**.
- These pages are stored at **consecutive virtual addresses** starting from **0x4000**.
- Since the system can hold only **8 pages** in physical memory, the rest will eventually be swapped out to disk.
- Each iteration accesses one page, writes data, and computes a checksum to confirm correct read/write behavior.
- The first half (iterations 0–24) follows a **sequential pattern**, ensuring every page is touched.
- The second half (25–49) uses a **random-like pattern** to stress-test MRU swapping.
- These are printed every 5 iterations:

- **Page Faults:** Count of times a page not in memory was accessed.
- **Swap-ins:** Pages brought from disk into RAM.
- **Swap-outs:** Pages evicted from RAM to disk.
- You can observe:
 - At iteration 4 → 7 faults (first few pages loaded).
 - At iteration 9 → 12 faults, 2 swap-outs (memory full, evictions begin).
 - By iteration 49 → 49 faults, 39 swap-outs — confirming active MRU replacement.
- **dumpmru()** displays the **order of pages in memory**.
- The **first entry** is the most recently accessed page (MRU).
- Whenever a new page is accessed, it moves to the top.
- Once 8 pages are in memory, adding a new one causes the **topmost (most recent)** page to be swapped out — the hallmark of MRU behavior.
- Indicates **frequent page replacements** due to limited physical memory.
- The difference between swap-ins and swap-outs arises from reused pages that remained in memory near the end.